MODULE 4

Systems & Projections

Linear Algebra III — the machinery behind fitting a model to data

Mathematical Foundations for Data Science & Computational Intelligence · Mohsin Raza · Fall 2026

SESSION 2

Computing it without breaking it

Callback to Day 1: floats aren't the reals

Day 1: integers in a computer aren't closed under +. Push far enough and they overflow.

```
> 0.1 + 0.2 == 0.3
[1] FALSE
```

Real numbers get the same treatment: a double stores about **16 significant digits** and rounds the rest.

Usually harmless. But suppose a calculation amplifies tiny input errors by a factor of 10^{12}…

How many trustworthy digits are left?

Condition number: how many digits you lose

κ(A) measures how much A amplifies small errors. Rule of thumb: **you lose about $\log_{10}$ κ digits.**

$$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}}, \qquad A^\top A = V\Sigma^2 V^\top \implies \kappa(A^\top A) = \kappa(A)^2$$

$\kappa(A) = 10^8$

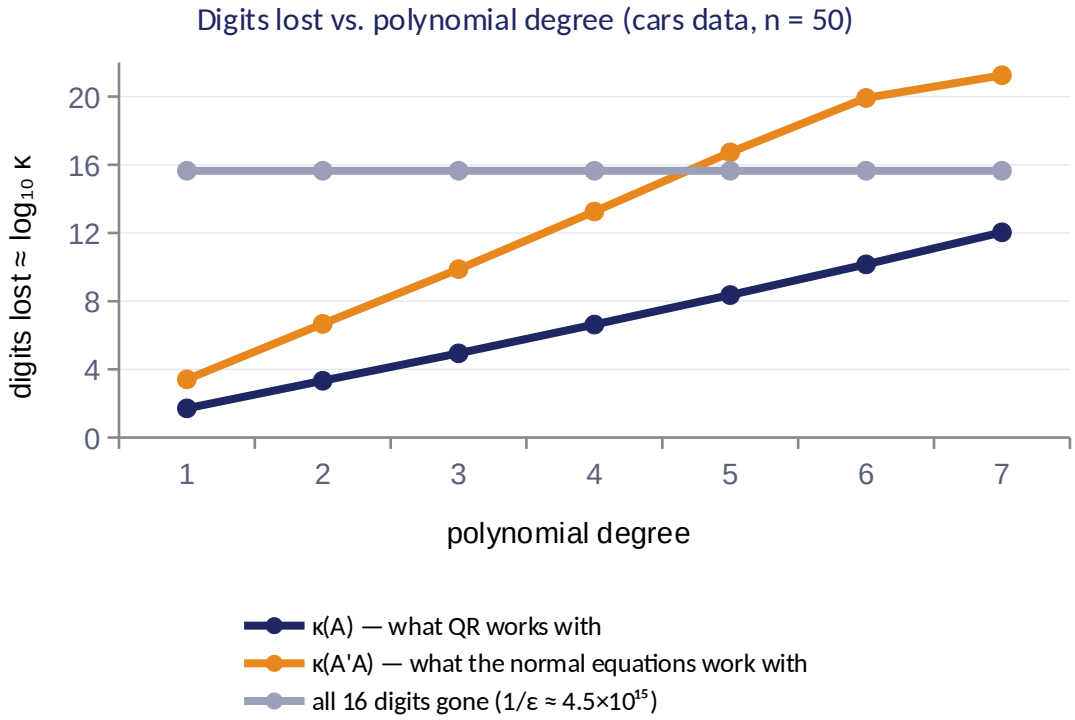
lose ~8 digits, keep ~8. Fine.

$\kappa(A^\top A) = 10^{16}$

lose ~16 digits, keep none.

The normal equations square the problem's sensitivity before solving it.

Real data: stopping distance vs. speed



Degree 5:
$\kappa(A) \approx 2.3 \times 10^{8}$
$\kappa(A^{T}A) \approx 5.3 \times 10^{16}$

Normal equations in R:
"system is computationally singular"

QR on the same data:
solves it — fitted values match a stable reference to ~10^{-11}.

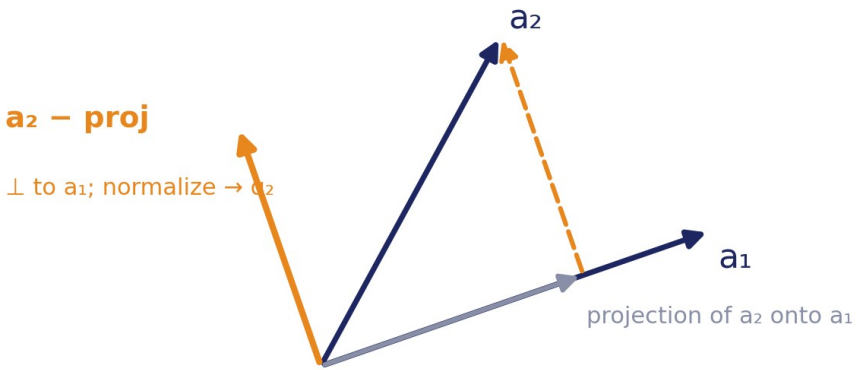
What A would make least squares trivial?

$$A = Q, \quad Q^\top Q = I \quad \Longrightarrow \quad \hat{\mathbf{x}} = (Q^\top Q)^{-1} Q^\top \mathbf{b} = Q^\top \mathbf{b}$$

If A's columns were orthonormal, there'd be nothing to invert and no κ^2 — just dot products.

Can we swap A's columns for orthonormal ones that span the same space?

Gram–Schmidt: project, subtract, repeat



1. Keep a_1; normalize it → q_1
2. From a_2, subtract its projection onto q_1 (Module 2)
3. Normalize what's left → q_2. Continue for a_3, a_4, …
4. The amounts you subtracted form an upper-triangular R

$$A = QR \implies A^\top A\,\hat{\mathbf{x}} = A^\top \mathbf{b} \text{ becomes } R\,\hat{\mathbf{x}} = Q^\top \mathbf{b}$$

Our example, through QR

$$\mathbf{q}_1 = \frac{1}{\sqrt{3}}\begin{bmatrix}1\\1\\1\end{bmatrix}, \quad \mathbf{a}_2 - (\mathbf{a}_2^\top \mathbf{q}_1)\,\mathbf{q}_1 = \begin{bmatrix}1\\2\\3\end{bmatrix} - \begin{bmatrix}2\\2\\2\end{bmatrix} = \begin{bmatrix}-1\\0\\1\end{bmatrix}$$

$$R = \begin{bmatrix}\sqrt{3} & 2\sqrt{3}\\ 0 & \sqrt{2}\end{bmatrix}, \quad Q^\top \mathbf{b} = \begin{bmatrix}14/\sqrt{3}\\ 3/\sqrt{2}\end{bmatrix} \xrightarrow{\text{back-substitute}} \hat{\mathbf{x}} = \begin{bmatrix}5/3\\ 3/2\end{bmatrix}$$

(−1, 0, 1) is s = (1, 2, 3) minus its mean. Centering a feature is one step of Gram–Schmidt against the intercept column.

Redundant columns: the SVD returns

$$A = U\Sigma V^\top \quad\Longrightarrow\quad \hat{\mathbf{x}} = A^{+}\mathbf{b} = V\,\Sigma^{+}U^\top\mathbf{b}, \qquad \Sigma^{+}:\ \sigma_i \mapsto \begin{cases} 1/\sigma_i & \sigma_i > \text{tol} \\ 0 & \text{otherwise} \end{cases}$$

A⁺ is the pseudoinverse *(the + is not a transpose): invert the singular values you trust, zero the rest.*

A classic production bug: car weight entered twice — in 1000 lb and in kg (mtcars). The columns are exactly proportional, so A is rank-deficient.

Normal equations	error: computationally singular
lm() (pivoted QR)	coefficient reported as NA
SVD pseudoinverse	minimum-norm solution; same predictions

singular values of A

8617

1.62

6×10^{-15}

The last direction carries no information → set 1/σ to 0.

Three solvers, different failure modes

	Normal equations	QR	SVD
Forms A^TA?	Yes	No	No
Sensitivity	$\kappa(A)^2$	$\kappa(A)$	$\kappa(A)$
Redundant columns	Fails	Detects (pivoting → NA)	Handles (minimum norm)
Relative cost	Cheapest	Moderate	Most expensive
You'll find it in	Textbooks, hand calculations	R's lm()	numpy.linalg.lstsq

Default: QR. Reach for SVD when features may be redundant. Use the normal equations on paper, not in production.

What this changes in your day-to-day work

Standardize features

Degree-5 fit on the cars data: $\kappa(A)$ drops from 2.3×10^8 to 62 after centering and scaling speed. Now even the normal equations cope.

Read NA as a message

An NA coefficient in lm() means two features carry the same information. Fix the data, don't fight the solver.

Too big to factor?

When A has millions of rows and columns, even QR is too expensive — you iterate instead. That's Module 8.

Where $\|Ax - b\|^2$ shows up next

Module 5

Multivariable calculus

Set the gradient of $\|Ax - b\|^2$ to zero — the same normal equations, by a second road.

Module 7

Regression & statistics

Add one term, $\lambda\|x\|^2$, and get ridge regression — which also cures bad conditioning and redundancy.

Module 8

Optimization

When A is too big to factor, walk downhill on $\|Ax - b\|^2$ instead: gradient descent.

PRACTICE

Module 4 notebook: Systems & Projections

Colab notebook · ~45–60 min · ungraded · every number from today's slides, reproducible

Next: Module 5 — Multivariable Calculus

Questions?

BACKUP A

Why $P = A(A^TA)^{-1}A^T$ really is a projection

$$P^2 = A(A^\top A)^{-1}\underbrace{A^\top A\,(A^\top A)^{-1}}_{=I}A^\top = P, \qquad P^\top = A\big((A^\top A)^{-1}\big)^\top A^\top = P$$

- $P^2 = P$: applying it twice changes nothing — the shadow of a shadow is itself.
- $P^T = P$: it projects straight down (orthogonally), not at an angle.
- $I - P$ projects onto the perpendicular direction: it gives the residual, $r = (I - P)b$.

BACKUP B

Why $\kappa(A^TA) = \kappa(A)^2$

$$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}}, \qquad A^\top A = V\Sigma^2 V^\top \implies \kappa(A^\top A) = \kappa(A)^2$$

- From Module 3: $A = U\Sigma V^T$, so $A^TA = V\Sigma U^TU\Sigma V^T = V\Sigma^2V^T$.
- So the singular values of A^TA are σ_i^2 — the largest and smallest both get squared.
- Their ratio squares: $\kappa(A^TA) = \sigma^2\text{max} / \sigma^2\text{min} = \kappa(A)^2$. QR works with R, which has the same singular values as A.

BACKUP C

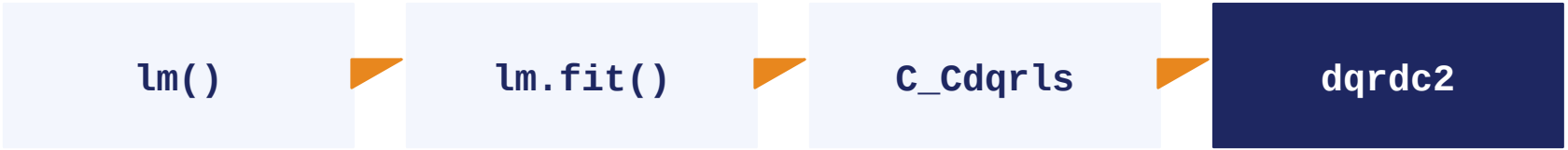
The calculus road (preview of Module 5)

$$f(\mathbf{x}) = \|A\mathbf{x} - \mathbf{b}\|^2 = \mathbf{x}^\top A^\top A\mathbf{x} - 2\mathbf{b}^\top A\mathbf{x} + \mathbf{b}^\top\mathbf{b}, \qquad \nabla f = 2A^\top(A\mathbf{x} - \mathbf{b}) = \mathbf{0}$$

- Setting the gradient to zero gives $A^TAx = A^Tb$ — the same normal equations the geometry gave us.
- The Hessian is $2A^TA$, which is positive semidefinite — so this critical point is a minimum, not a maximum.
- Two independent arguments, one answer: that's usually a sign you've found the right idea.

BACKUP D

Inside R's lm()



- lm.fit() supports only method = "qr". It never forms X^TX.
- dqrdc2 is R's modified version of LINPACK's Householder QR, with limited column pivoting: columns with near-zero norm (after orthogonalizing) are moved to the end.
- Those moved columns are 'aliased' — which is why their coefficients print as NA.
- Source: R's own documentation, ?qr and the body of lm.fit.